QLens · Viva Cheatsheet

QLens: An Automated Relational Database Design and Normalization System

Aditya Gaur · Aditya Choudhary | CSC2103 Relational Database Management Systems · B.Tech CSE, Semester III · Manipal University Jaipur

How to read. Each box: **What** (one-line definition) · **In QLens** (screen + file) · a real snippet. Python paths are under `backend/qlens/`, e.g. `design/ddl.py:_create_table L86`. SQL migrations are cited as `001:212` = `backend/sql/migrations/001_schema.sql` line 212. `ws_20` = workspace number 20 (any id works). Snippets are copied from the code; long lines are re-wrapped and `...` marks lines left out. **Areas:** Intro & ER · Relational model · Relational algebra · SQL · Advanced SQL · Normalization · Transactions · Security · Optional / code map

PART A RDBMS topics in QLens

one box per syllabus topic

File system vs DBMS: the spreadsheet problem

What. A spreadsheet (flat file) repeats the same fact on many rows and checks nothing. A DBMS stores each fact once, enforces rules (constraints), controls concurrent users and recovers after a crash.

Example. The **fee register** sample: 3,000 rows = 300 students × 10 months, columns Student Name, Student Phone, Batch, Teacher, Teacher Phone, Month, Fee Paid, Payment Mode.

- **Update anomaly:** a teacher's phone is copied on every row of the batch. The sample has Mr. Gupta's *old* number in exactly 5 rows: one fact, two values.
- **Insertion anomaly:** a new batch and its teacher cannot be stored until some student of that batch has a fee row.
- **Deletion anomaly:** deleting the last fee rows of a batch also loses who teaches it.

In QLens. Import keeps the sheet as it is (`ws_<id>_src._raw`); Analyze finds the repetition (functional dependencies); Design splits it into `teachers`, `batches`, `students`, `fee_register`, so each fact is stored once.

Schema, instance, data independence

What. **Schema** = the design (tables, columns, types, constraints), changes rarely. **Instance** = the rows at one moment. **Data independence** = changing one level does not break the level above it.

- A design (schema) is stored as *data* in `qlens.schema_proposals`, `proposed_tables`, `proposed_columns`; the instance is the rows of `ws_20.students` etc.
- The **system catalog** (data dictionary) is read by `design/catalog.py:describe L56`: `pg_class`, `pg_attribute`, `pg_constraint`, `pg_index`.
- Three levels: external = views (`original_sheet`, `v_workspace_summary`); conceptual = tables; physical = heap files and B-tree indexes.
- **Logical** independence: after Apply the sheet is 4 tables, yet `SELECT * FROM original_sheet` still returns the old sheet columns.
- **Physical** independence: "Explain & speed up" adds an index; the same SELECT text just runs faster.

DBMS architecture and users

What. Three-tier: client → application server → database server. **In QLens.** Browser (React) → FastAPI (Python, `/api/...`) → PostgreSQL 18 (diagram in Part B). Users: **naive** users click through Import/Analyze/Design; **sophisticated** users type SQL in Data > SQL; **application programmers** (us) wrote every SQL statement by hand (no ORM); the **DBA** step is `setup_db.py` (creates role `qlens_app`, the database and the `pg_trgm` extension) and `migrate.py` (runs `000`–`008`, records them in `qlens.schema_migrations`). Applications: fee registers, shop bills, hostel mess (the three samples in `samples/generate.py`).

ER model: entities, attributes, relationships

What. Entity set (rectangle), attribute (ellipse; key underlined), relationship set (diamond), cardinality 1:1, 1:N, M:N. Attributes: simple/composite, single/multivalued, stored/derived. **In QLens.** The `qlens` schema itself: User *owns* (1:N) Workspace *holds* (1:1) Dataset *has* (1:N) Column; Functional Dependency *LHS* (M:N) Column; Dataset *proposes* (1:N) Schema Proposal *contains* (1:N) Proposed Table. Sketch in Part B.

- **Derived attribute**, computed by PostgreSQL (`001:136`):

```
is_exact        boolean GENERATED ALWAYS AS (violating_rows = 0) STORED,
violation_ratio numeric GENERATED ALWAYS AS
    (violating_rows::numeric / NULLIF(row_count, 0)) STORED,
```

- **Multivalued attribute**: an FD's left side can have 2 columns → rows of `fd_lhs_columns`, never a list inside one cell.
- **Weak entity**: `duplicate_values(group_id, value)` has no own id; partial key `value` + owner `duplicate_groups` (`001:172`).
- **Specialization** as a type attribute: `proposed_tables.kind IN ('entity','fact')`, `proposed_columns.role IN ('pk','fk','attribute')`.

ER → relational mapping (the 4 rules)

1. Strong entity set → table, its key → `PRIMARY KEY`: `users(id, email, ...)` (`001:33`).
2. 1:N → foreign key on the N side: `workspaces.owner_id REFERENCES users(id)` (`001:61`).
3. 1:1 → foreign key + `UNIQUE`: `datasets.workspace_id ... UNIQUE` (`001:72`).
4. M:N → its own table, primary key = both foreign keys:

```
CREATE TABLE qlens.fd_lhs_columns (                          -- 001:148
    fd_id     bigint NOT NULL REFERENCES
              qlens.functional_dependencies (id) ON DELETE CASCADE,
    column_id bigint NOT NULL REFERENCES
              qlens.dataset_columns (id) ON DELETE CASCADE,
    PRIMARY KEY (fd_id, column_id)
);
```

Also M:N: `candidate_key_columns`, `proposed_unique_key_columns`. Result: 26 tables in schema `qlens` (+ `schema_migrations`). The user's data follows the same rules: Teacher 1:N Batch 1:N Student 1:N fee row → `batches.teacher_id`, `students.batch_id`, `fee_register.student_id`.

Relation, tuple, attribute, domain

What. Relation = table, tuple = row, attribute = column, **domain** = set of allowed values (type + CHECK). Degree = number of columns; cardinality = number of rows. **In QLens.** On Import the database itself decides each column's domain: `ingest/stage.py:_infer_type L97` picks the first of integer, numeric, boolean, date that fits every non-empty value, else text:

```
SELECT bool_and({c} ~ %(int)s) AS is_int,
       bool_and({c} ~ %(num)s) AS is_num,
       bool_and(lower({c}) = ANY(%(bools)s)) AS is_bool,
       bool_and(CASE WHEN {c} ~ %(dateish)s
           THEN qlens.try_date({c}) IS NOT NULL ELSE false END)
       AS is_date
FROM {t} WHERE {c} IS NOT NULL
```

Then `_build_staging L140` runs `CREATE TABLE _staging AS SELECT _row_id, ...` with casts (`fee_paid::bigint`). Phones like "+91 98765 43210" stay `text`. A domain as a CHECK: `inferred_type IN ('integer','numeric','boolean','date','text')` (`001:91`).

Keys: super, candidate, primary, foreign; surrogate vs natural

What. **Superkey**: columns whose values differ in every row. **Candidate key**: a minimal superkey. **Primary key**: the chosen one (others = alternate keys). **Foreign key**: refers to a key of another table. **Surrogate** key = a number made by the system (`id`); **natural** key = from the data (`email`, `student_name`). **In QLens.** Analyze → `analysis/discover.py:_find_keys L69` tries 1-, 2-, 3-column sets:

```
if any(set(k) <= set(names) for k in keys):
    return False   # a superset of a smaller key is not a new key
if any(c.null_count > 0 for c in combo):
    return False   # a key must be filled in on every row
if math.prod(c.distinct_count for c in combo) < row_count:
    return False   # cannot be unique: never reaches the database
return bool(conn.execute(
    "SELECT qlens.is_unique(%s, '_staging', %s) AS u",
    (schema, list(names))).fetchone()["u"])
```

Fee register: candidate key e.g. {student_name, month}. Generated tables get a surrogate `id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY` and keep the natural key as `UNIQUE` (`design/ddl.py:_create_table L90, L98`).

Integrity constraints

What. Rules the DBMS enforces on every change: domain (type, CHECK), entity integrity (primary key is unique and not null), key (UNIQUE), referential integrity (FOREIGN KEY).

```
email text NOT NULL UNIQUE                                    -- 001:36
      CHECK (email = lower(email) AND position('@' in email) > 1),
UNIQUE (owner_id, name)                                       -- 001:66
CHECK ((status = 'ok') = (error_message IS NULL))             -- 001:313
CHECK (status <> 'applied' OR lossless_ok IS TRUE)            -- 001:212
```

- **Why `IS TRUE`**: a CHECK *passes* when it is NULL (unknown). `lossless_ok` is NULL until the check runs; `lossless_ok = true` would let an unchecked design become "applied". `NULL IS TRUE` is false, so it is refused.
- **`UNIQUE NULLS NOT DISTINCT`** on generated tables (`ddl.py L98`): two NULLs normally never clash; here an empty natural key (no teacher) is one value, stored once.
- **FK actions**: `ON DELETE CASCADE` deleting a user deletes their workspaces, datasets… (`001:61`); `SET NULL` deleting a saved query keeps its run history (`001:303`); `RESTRICT` on generated FKs: a student with fee rows cannot be deleted (`ddl.py L102`).

Relational algebra: the "Step through" button

What. Unary: σ selection (rows), π projection (columns), ρ rename. Binary: $\times$ product, $\bowtie$ join, $\cup$, $\cap$, $-$. Extended: γ grouping, δ duplicate elimination, τ sort. **In QLens.** Data > SQL > Step through → `stepthrough/stages.py:_build L208` cuts one SELECT into a chain of runnable SELECTs, one per operator, runs each in the sandbox and shows the rows after each step:

FROM …JOIN	$\bowtie$ join	GROUP BY	γ grouping
FROM a, b	$\times$ product	HAVING	σ on groups
WHERE	σ selection	SELECT	π projection
DISTINCT	δ	ORDER BY/LIMIT	τ sort

```
SELECT b.batch, count(*) AS unpaid
FROM fee_register f JOIN students s ON s.id = f.student_id
                    JOIN batches  b ON b.id = s.batch_id
WHERE f.fee_paid = 0
GROUP BY b.batch HAVING count(*) > 25;
```

$= \pi_{batch,\ unpaid}\left(\sigma_{count>25}\left(\gamma_{batch;\ count(*)}\left(\sigma_{fee_paid=0}(F \bowtie S \bowtie B)\right)\right)\right)$.
UNION/INTERSECT/EXCEPT are refused here (`stages.py:_parse`): they combine two whole queries.

Set difference: EXCEPT ALL in the lossless check

What. $r - s$ = tuples of r that are not in s. SQL `EXCEPT` is set difference; `EXCEPT ALL` is the multiset (bag) version that also counts repeated rows. **In QLens.** Apply → `design/ddl.py:_verify L220` (schemas left out; `cols` = the 8 sheet columns):

```
SELECT (SELECT count(*) FROM _staging)       AS staging_rows,
       (SELECT count(*) FROM original_sheet) AS reconstructed_rows,
       (SELECT count(*) FROM (SELECT cols FROM original_sheet
            EXCEPT ALL SELECT cols FROM _staging) d)       AS
     ↪ extra_rows,
       (SELECT count(*) FROM (SELECT cols FROM _staging
            EXCEPT ALL SELECT cols FROM original_sheet) d) AS
     ↪ missing_rows
```

Both differences 0 ⇒ joining the new tables gives back exactly the sheet. **Why.** `ALL`: with plain EXCEPT a row that is in the sheet twice but rebuilt once would go unnoticed.

DDL: CREATE / ALTER / DROP

In QLens.

- `CREATE SCHEMA ws_20, ws_20_src, ws_20_lab` and `DROP SCHEMA … CASCADE`: `db.py:create_ws_schema`, `drop_ws_schema`.
- `CREATE TABLE … AS SELECT` (typed copy) + `ALTER TABLE … ADD PRIMARY KEY`: `ingest/stage.py:_build_staging L140`.
- The design's script (Design > SQL panel): `design/ddl.py:render L235` = schema, tables (referenced tables first), loads, triggers, view, verify. Real output for the fee register:

```
CREATE TABLE ws_20.fee_register (
    id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    student_id bigint NOT NULL,
    month text NOT NULL,
    fee_paid bigint NOT NULL,
    payment_mode text,
    CONSTRAINT fee_register_student_id_month_key
        UNIQUE NULLS NOT DISTINCT (student_id, month),
    CONSTRAINT fee_register_student_id_fkey FOREIGN KEY (student_id)
        REFERENCES ws_20.students (id) ON DELETE RESTRICT
);
```

`CREATE VIEW original_sheet` (`ddl.py:_view L159`); `CREATE INDEX` (`perf/lab.py:try_index L239`); `CREATE TRIGGER` (`ddl.py:_audit_trigger L107`).

DML: INSERT, UPDATE, DELETE, COPY

In QLens.

- Import: `COPY ws_20_src._raw (...) FROM STDIN` (`ingest/stage.py:92`), then `INSERT INTO qlens.datasets / dataset_columns`.
- Apply fills each table with `INSERT … SELECT` from `_staging` (`design/ddl.py:_load L121`); entity tables use `DISTINCT`; a foreign key is looked up by joining the parent on its natural key:

```
INSERT INTO ws_20.students (student_name, student_phone, batch_id)
SELECT DISTINCT s.student_name, s.student_phone, batches.id
FROM ws_20_src._staging s
LEFT JOIN ws_20.batches ON batches.batch = s.batch
ORDER BY 1, 2, 3
```

- Clean-up: `UPDATE _staging` (`analysis/cleanup.py:_rewrite L219`).
- Data > Browse: `data/service.py:insert_row L407`, `update_row L420`, `delete_row L435`, e.g. `UPDATE {} SET {} WHERE id = %s RETURNING {}` (each SET part is `{} = %s`).
- Data > SQL: one typed INSERT/UPDATE/DELETE (`query/routes.py:run_write L188`).

DCL: roles, GRANT, REVOKE

What. DCL gives or takes away privileges. **In QLens.** `workspaces/service.py:create_workspace L71` and `db.py:create_ws_schema`, `grant_ws_writes`; shown as the SQL they send for workspace 20:

```
CREATE ROLE qlens_ws_20 LOGIN PASSWORD '...'
    NOINHERIT CONNECTION LIMIT 3;
ALTER ROLE qlens_ws_20 SET statement_timeout = '5s';
ALTER ROLE qlens_ws_20 SET default_transaction_read_only = on;
GRANT USAGE ON SCHEMA ws_20 TO qlens_ws_20;
GRANT SELECT ON ALL TABLES IN SCHEMA ws_20 TO qlens_ws_20;
GRANT INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA ws_20
    TO qlens_ws_20;                          -- main schema only
REVOKE INSERT, UPDATE, DELETE ON ws_20.original_sheet
    FROM qlens_ws_20;                        -- views stay read-only
REVOKE EXECUTE ON FUNCTION qlens.log_row_change()
    FROM PUBLIC;                             -- 007:9
```

`ALTER DEFAULT PRIVILEGES` gives tables created later the same rights. The role's password is derived (HMAC of the secret key), never stored (`db.py:ws_password`).

Joins

What. Inner join keeps matching rows; left outer join keeps every left row (NULLs when no match); self join joins a table with itself; lateral join lets the right side use the left row. **In QLens.** Reading an FD with its column names, `analysis/discover.py:_FD_SQL L197`:

```
SELECT f.id, rc.name AS rhs,
       array_agg(lc.name ORDER BY lc.ordinal) AS lhs, ...
FROM qlens.functional_dependencies f
JOIN qlens.dataset_columns rc ON rc.id = f.rhs_column_id
JOIN qlens.fd_lhs_columns l   ON l.fd_id = f.id
JOIN qlens.dataset_columns lc ON lc.id = l.column_id
WHERE f.dataset_id = %(ds)s
GROUP BY f.id, rc.name, rc.label, rc.ordinal
```

- `original_sheet`: `fee_register LEFT JOIN students LEFT JOIN batches LEFT JOIN teachers` (`ddl.py:_view`).
- Self join: `FROM c a JOIN c b ON a.norm = b.norm AND a.val < b.val` pairs two spellings of the same value (`cleanup.py:_column_clusters L115`).
- `LEFT JOIN LATERAL` in `v_workspace_summary` (`002:70`).

Subqueries

What. A query inside a query: scalar (one value), in `IN`/`EXISTS`, in `FROM` (derived table), correlated (uses the outer row); `WITH` names one (CTE). **In QLens.**

- Scalar + derived-table subqueries: the lossless check above; `is_unique`: `SELECT count(*) FROM (SELECT DISTINCT cols FROM t) d` (`004:43`).
- `IN` subquery: Browse search finds a fee row by its student's name (`data/service.py:_view L179`): `t.student_id IN (SELECT r.id FROM ws_20.students r WHERE <name> ILIKE %s)`.
- CTEs: `check_fd`, `v_slow_queries`, `cleanup._rewrite`.
- Correlated `NOT EXISTS`, to run in Data > SQL (students who never missed a payment):

```
SELECT s.student_name FROM students s
WHERE NOT EXISTS (SELECT 1 FROM fee_register f
                  WHERE f.student_id = s.id AND f.fee_paid = 0);
```

Views

What. A virtual table: a stored query, no stored rows. **In QLens.** `v_workspace_summary` (`002:54`, dashboard cards), `v_slow_queries` (`002:84`, History: runs above the workspace's 90th percentile via `percentile_cont(0.9)`), `v_saved_query_trend` (`002:112`, window functions `row_number()`, `lag()`, `avg() OVER (... ROWS BETWEEN 4 PRECEDING AND CURRENT ROW)`). Per workspace, `design/ddl.py:_view L159` rebuilds the sheet:

```
CREATE VIEW ws_20.original_sheet AS
SELECT students.student_name, students.student_phone, batches.batch,
       teachers.teacher, teachers.teacher_phone, fee_register.month,
       fee_register.fee_paid, fee_register.payment_mode
FROM ws_20.fee_register
LEFT JOIN ws_20.students ON students.id = fee_register.student_id
LEFT JOIN ws_20.batches ON batches.id = students.batch_id
LEFT JOIN ws_20.teachers ON teachers.id = batches.teacher_id
```

Aggregation: GROUP BY, HAVING

What. Aggregate functions (`count`, `sum`, `max`, `avg`) summarize each group; `HAVING` filters whole groups. **In QLens.** The FD test `qlens.check_fd` (`004:14`) is two GROUP BYs (placeholders `%1$s`, `%2$I` written as `lhs`, `rhs`):

```
WITH c AS (SELECT lhs, rhs, count(*) AS cnt
           FROM t GROUP BY lhs, rhs),
     g AS (SELECT lhs, sum(cnt) AS n, max(cnt) AS keep
           FROM c GROUP BY lhs)
SELECT (SELECT count(*) FROM t),              -- row_count
       count(*),                              -- lhs_values
       count(*) FILTER (WHERE n >= 2),        -- repeated_groups
       coalesce(sum(n - keep), 0)::bigint     -- violating_rows
FROM g
```

Example. teacher → teacher_phone: in Mr. Gupta's group n = all his rows, `keep` = rows with the common number, so $n - keep = 5$ (the old number) ⇒ 5 violating rows. batch → teacher: every group has one teacher ⇒ 0 (exact).

Triggers (OLD / NEW)

What. A procedure the DBMS runs by itself on INSERT/UPDATE/DELETE: BEFORE or AFTER, FOR EACH ROW; `OLD` = row before, `NEW` = row after.

1. `touch_updated_at`: BEFORE UPDATE, `NEW.updated_at := clock_timestamp()` (`002:11`; on workspaces, datasets, saved_queries, assistant_conversations).
2. `record_query_run`: AFTER INSERT ON query_runs `WHEN (NEW.saved_query_id IS NOT NULL AND NEW.status = 'ok')` → `run_count + 1` (`002:29–43`).
3. `log_row_change`: audit log of every change on the generated tables (`006:24`), attached by Apply *after* the loads (`ddl.py:_audit_trigger L107`):

```
IF TG_OP IN ('UPDATE', 'DELETE') THEN old_row := to_jsonb(OLD); END IF;
IF TG_OP IN ('INSERT', 'UPDATE') THEN new_row := to_jsonb(NEW); END IF;
INSERT INTO qlens.row_changes
       (workspace_id, table_name, op, row_pk, before, after)
VALUES (ws_id, TG_TABLE_NAME, TG_OP,
        (coalesce(new_row, old_row) ->> 'id')::bigint,
        old_row, new_row);
RETURN NULL;   -- the result of an AFTER trigger is ignored
-- attached to each generated table:
CREATE TRIGGER log_row_change AFTER INSERT OR UPDATE OR DELETE
    ON ws_20.students FOR EACH ROW
    EXECUTE FUNCTION qlens.log_row_change();
```

Why. a trigger: it catches every change (Browse forms, typed SQL, anything) inside the same transaction; no code path can forget it. `SECURITY DEFINER` (`007:7`) lets it write `qlens.row_changes` even when the sandbox role made the change.

Cursors

What. A cursor points into a query's result so rows can be read a few at a time: `DECLARE`, `FETCH`, `CLOSE` (or end of transaction). **In QLens.** Every SELECT from Data > SQL, each step-through stage and Ask AI's `run_sql` go through `query/sandbox.py:run`:

```
_CURSOR = "qlens_run"
_DECLARE = f"DECLARE {_CURSOR} NO SCROLL CURSOR WITHOUT HOLD FOR "
...
conn.execute(_DECLARE + sql_text, prepare=True)
cur = conn.execute(f"FETCH FORWARD {int(max_rows) + 1} FROM {_CURSOR}")
```

It fetches 1,001 rows: 1,000 are shown, the extra one only says "truncated". A huge result never sits in memory. `NO SCROLL` = forward only; `WITHOUT HOLD` = closed when the (rolled-back) transaction ends. PL/pgSQL also uses an implicit cursor: `FOR t IN SELECT ... LOOP` in `006:47–61`.

Stored procedures / functions (PL/pgSQL)

What. A named program stored in the database and called from SQL; PL/pgSQL adds variables, IF, loops and exception handling. (In PostgreSQL these are functions, called with SELECT.) **In QLens.** `check_fd(schema, table, lhs text[], rhs) RETURNS TABLE (...)` (`004:14`) and `is_unique(schema, table, cols text[]) RETURNS boolean` (`004:33`), both called from `analysis/discover.py`; and `try_date` (`003:8`):

```
CREATE FUNCTION qlens.try_date(p text) RETURNS date
LANGUAGE plpgsql STABLE AS $$
BEGIN
    IF p ~ '^\d{4}-\d{2}-\d{2}$' THEN
        RETURN p::date;
    ELSIF p ~ '^\d{1,2}[/-]\d{1,2}[/-]\d{4}$' THEN
        RETURN to_date(replace(p, '-', '/'), 'DD/MM/YYYY');
    END IF;
    RETURN NULL;
EXCEPTION WHEN others THEN
    RETURN NULL;          -- 31/02/2026 gives NULL, not an error
END;
$$;
```

Dynamic SQL is safe: table and column names go through `format('%I')` (quoted identifiers).

Indexing (B-tree) and "Explain & speed up"

What. An index (PostgreSQL's default is a B$^+$-tree, called `btree`) finds rows without scanning the whole table; it costs space and slows writes. PRIMARY KEY/UNIQUE create one automatically; others are **secondary** indexes. Each one in `001` has a comment naming the query it serves:

```
CREATE INDEX query_runs_workspace_time_idx              -- 001:317
    ON qlens.query_runs (workspace_id, created_at DESC); -- History
CREATE INDEX query_runs_saved_query_time_idx            -- 001:320
    ON qlens.query_runs (saved_query_id, created_at)
    WHERE saved_query_id IS NOT NULL;                   -- partial
CREATE INDEX sessions_user_id_idx ON qlens.sessions (user_id);
                  -- 001:53, FK column: serves ON DELETE CASCADE
```

PostgreSQL does not index foreign keys by itself, so 001 adds them. A **GIN** trigram index finds look-alike spellings: `CREATE INDEX ON pg_temp._qlens_vals USING gin (val gin_trgm_ops)` (`cleanup.py:147`).

In QLens. Data > SQL > Explain & speed up → `perf/lab.py:try_index L205`: median of 5 `EXPLAIN ANALYZE` runs → `CREATE INDEX` on a lab copy → `ANALYZE` → 5 runs again → row in `qlens.index_experiments` (`speedup` = before/after, generated column). **Example.** `SELECT * FROM fee_register WHERE fee_paid = 2100`: Seq Scan before, index on `(fee_paid)` after.

Functional dependencies (exact vs approximate)

What. $X \to A$: any two rows with the same X have the same A (trivial if $A \in X$). **Exact**: holds in every row. **Approximate**: broken in a few rows (QLens: at most 2%, `APPROX_THRESHOLD = 0.02`, `discover.py:16`), usually a typing mistake. **In QLens.** Analyze → `analysis/discover.py:_find_fds L97` runs `check_fd` for single columns, then pairs:

```
m = conn.execute("SELECT * FROM qlens.check_fd(%s, '_staging', %s, %s)",
                 (schema, names, rhs)).fetchone()
if m["repeated_groups"] >= 2 and \
   m["violating_rows"] <= APPROX_THRESHOLD * m["row_count"]:
    return {"lhs": names, "rhs": rhs, **m}
```

Example. batch → teacher (exact); student_phone → student_name (approximate, 8 rows such as "riya sharma"); teacher → teacher_phone (approximate, 5 rows). The user fixes them (UPDATE to the most common value, `cleanup.py:fix_fd L312`) or rejects them (`accepted = false`). Measures (Fee Paid) and yes/no columns are never a left side (`discover.py:lhs_eligible L44`).

Attribute closure X^+ and candidate keys

What. X^+ = every attribute that X determines: repeat "if $Y \to A$ and $Y \subseteq$ result, add A". X is a superkey iff X^+ = all attributes. **In QLens.** `analysis/fd_algebra.py:closure L22`:

```
def closure(attrs, fds):
    result = set(attrs)
    changed = True
    while changed:
        changed = False
        for fd in fds:
            if fd.rhs not in result and fd.lhs <= result:
                result.add(fd.rhs)
                changed = True
    return frozenset(result)
```

Example. $\{batch\}^+ = \{batch, teacher, teacher_phone\}$. $\{student_name\}^+$ adds student_phone, batch, teacher, teacher_phone but never month, so the key needs month: {student_name, month} is a candidate key. QLens finds keys from the data (`is_unique`); closure is used for the canonical cover and 3NF.

Canonical cover F_c

What. Smallest set of FDs with the same closure as F: (1) one attribute on each right side, (2) no extraneous attribute on a left side, (3) no redundant FD. **In QLens.** `analysis/fd_algebra.py:minimal_cover L34` (docstring: "Canonical cover"):

```
# Left-reduce: drop an LHS attribute when the rest still
# implies the RHS.
if len(lhs) > 1 and fd.rhs in closure(lhs - {a}, work):
    lhs.discard(a)
...
# Remove redundant FDs, scanning last to first.
if fd.rhs in closure(fd.lhs, rest):
    deduped = rest
```

Example. F_c of the cleaned fee register: student_name → student_phone, batch; student_phone → student_name; batch → teacher; teacher → teacher_phone; teacher_phone → teacher. Dropped: student_name → teacher (follows by transitivity); the Analyze screen marks it "derived" (`discover.py:load_fds L211`).

Normal forms: 1NF, 2NF, 3NF, BCNF

What. **1NF**: atomic values. **2NF**: no non-prime attribute depends on *part* of a candidate key. **3NF**: for every $X \to A$, X is a superkey or A is prime (no transitive dependency). **BCNF**: every determinant is a superkey. 4NF/5NF deal with multivalued and join dependencies (QLens finds only FDs). **Example.** The fee register sheet, key {student_name, month}:

- 1NF ✓ (one value per cell).
- 2NF ×: student_phone and batch depend on student_name alone (part of the key).
- 3NF ×: student_name → batch → teacher → teacher_phone (transitive).
- After Apply all four tables are in **BCNF**: each determinant is a key of its table (student_name and student_phone are both UNIQUE in `students`; teacher and teacher_phone in `teachers`).

QLens's own schema is mostly BCNF; deliberate exceptions keep values correct by other means: `dataset_id` copied into `functional_dependencies` (saves a join), `is_exact` (generated column), `run_count` (kept by a trigger).

3NF decomposition algorithm

What. (Korth) 1. compute F_c; 2. one relation XY for each $X \to Y$ in F_c; 3. if no relation contains a candidate key, add one that does; 4. remove a relation contained in another. The result is in 3NF, lossless and dependency preserving. **In QLens.** Design → `design/proposal.py:build L186` calls `analysis/fd_algebra.py:synthesize_3nf L69`: `cover = minimal_cover(...)` (L77), group by left side (L79), merge equivalent left sides such as student_name ↔ student_phone (L89), add the fact relation with the key (L135–160), drop contained relations (L162). **Example.** R_1(student_name, student_phone, batch), R_2(batch, teacher), R_3(teacher, teacher_phone), R_4(student_name, month, fee_paid, payment_mode) ← holds the key. Then `proposal.py:_plan L96` names them `students`, `batches`, `teachers`, `fee_register`, adds `id`, keeps the determinant as UNIQUE, and replaces a column that is another table's key by a foreign key (batch → `batch_id`, student_name → `student_id`).

Lossless join and dependency preservation

What. **Lossless**: joining the parts gives back exactly r, no extra (spurious) rows. Test for two parts: $R_1 \cap R_2 \to R_1$ or $R_1 \cap R_2 \to R_2$. **Dependency preserving**: every FD can be checked inside one table, $(F_1 \cup \cdots \cup F_n)^+ = F^+$. **Example.** fee_register ∩ students = the student (`student_id` = `students.id`), a key of students ✓. Each FD of F_c lies in one table and is enforced there: `UNIQUE (teacher)` of `teachers` enforces teacher → teacher_phone. **In QLens.** Apply also proves it on the real rows (EXCEPT ALL box) and refuses otherwise (`design/apply.py:90`):

```
mismatch = verify["extra_rows"] + verify["missing_rows"]
if mismatch:
    raise ApiError(409, "lossless_check_failed", ...)   # -> ROLLBACK
```

Transactions and ACID

What. A transaction runs all its statements or none. **A**tomicity, **C**onsistency, **I**solation, **D**urability.

- Every API request is one transaction: `db.py:get_conn` commits on success, rolls back on any error.
- Apply (`design/apply.py:apply_proposal L67`, with `conn.transaction()`: L70): drop old tables, create, load, triggers, view, lossless check. Check fails → exception → ROLLBACK, the old tables are untouched (**A**). Import is one transaction too (`ingest/stage.py:66`).
- A nested `conn.transaction()` is a **SAVEPOINT**: a refused Browse edit undoes only that statement (`data/service.py:_guarded L447`):

```
try:
    # A savepoint: after a failure the request's transaction
    # is still usable to explain it.
    with conn.transaction():
        return run()
except (psycopg.errors.IntegrityError, psycopg.errors.DataError) as e:
    raise _explain(conn, t, all_, op, row_id, values, e) from None
```

C: constraints are checked before COMMIT. **I**: row locks and snapshots (next box). **D**: PostgreSQL's write-ahead log (log-based recovery, checkpoints) keeps committed data after a crash; QLens relies on it.

Read-only transactions and time limits (sandbox)

In QLens. `query/sandbox.py:_begin` for every user SELECT:

```
conn.read_only = True  # the transaction opens as BEGIN READ ONLY
conn.execute("SELECT set_config('statement_timeout', %s, true), ...",
             (str(max(1, min(int(timeout_ms), MAX_TIMEOUT_MS))),))
...
finally:
    try:
        conn.rollback()          # a SELECT never commits anything
```

`sandbox.py:run_write`: `SET TRANSACTION READ WRITE` → one INSERT/UPDATE/DELETE → COMMIT, or ROLLBACK if PostgreSQL refuses it (SQLSTATE class 23 = integrity, 22 = data). Role defaults: `default_transaction_read_only = on`, `statement_timeout = '5s'`, `idle_in_transaction_session_timeout = '10s'`. A query past its limit is cancelled (SQLSTATE 57014) and stored as a `timeout` run.

Concurrency control: locks

What. Concurrent transactions must give a **serializable** result. Lock-based protocols: shared (read) and exclusive (write) locks; strict two-phase locking holds them until commit. A **deadlock** is a cycle of waits; PostgreSQL detects it and aborts one transaction. Plain reads use multiversion snapshots (MVCC), so readers do not block writers. **In QLens.**

- `SELECT ... FROM qlens.schema_proposals p ... FOR UPDATE OF p` (`design/apply.py:42`): two Apply clicks on one draft run one after the other; the second sees `'applied'` and gets 409.
- `FOR UPDATE` on staging rows before a clean-up UPDATE (`cleanup.py:225`); renames lock the row (`proposal.py:377`).
- `SELECT 1 FROM qlens.workspaces WHERE id = %s FOR NO KEY UPDATE` (`perf/lab.py:76`): one lab change at a time per workspace, without blocking inserts that reference it.
- Partial unique indexes (`001:217–221`): at most one `draft` and one `applied` design per dataset, even in a race; Apply marks the old one `superseded` first (`apply.py:104`).

```
CREATE UNIQUE INDEX schema_proposals_one_draft_idx
    ON qlens.schema_proposals (dataset_id) WHERE status = 'draft';
```

Security: roles and least privilege

What. Least privilege: every user and program gets only the rights it needs.

- App role `qlens_app` (`setup_db.py:52`): `LOGIN CREATEROLE`, owns schema `qlens`, not a superuser; `REVOKE ALL ON SCHEMA qlens FROM PUBLIC`.
- Each workspace has role `qlens_ws_<id>`: sees only `ws_<id>`, `_src`, `_lab`; no USAGE on `qlens` or other workspaces; may write only the generated tables.
- User SQL runs on a separate connection *as that role* (`query/sandbox.py:_connect`), so even SQL that slips past the checks cannot touch other data.
- Passwords: argon2 hash (`auth/service.py:signup`); only the SHA-256 of the session token is stored (`sessions.token_hash`, `001:46`).
- Someone else's workspace gives 404: `WHERE id = %s AND owner_id = %s` (`auth/deps.py:owned_workspace`).

Stopping SQL injection

What. SQL injection = user text becoming part of the SQL command. Rule: never paste input into SQL text.

1. **Values** → bound parameters `%s`, sent separately from the SQL (`auth/service.py:authenticate`). Typing `x' OR '1'='1` is just an email nobody has.

```
conn.execute("SELECT id, email, display_name, password_hash "
             "FROM qlens.users WHERE email = %s", (email,))
```

2. **Names** from the sheet → `sql.Identifier`, always double-quoted: `sql.SQL("UPDATE {} SET {} WHERE id = %s RETURNING {}")`, each `{}` filled with an `sql.Identifier` (`data/service.py:update_row L426`); names must also match `^[a-z_][a-z0-9_]{0,62}$` (CHECK in `001`). In PL/pgSQL: `format('%I.%I', ...)` (`004:18`).
3. **Typed SQL** (Data > SQL): `query/validate.py:validate_select L135` parses it with PostgreSQL's own parser (pglast): exactly one statement, must be a SELECT, no `SELECT INTO`, no `FOR UPDATE`, no INSERT/UPDATE/DELETE inside WITH, blocked functions (`pg_read_file`, `dblink`, `set_config`, ...). `prepare=True` makes PostgreSQL refuse a second statement. The role's privileges remain the real boundary.

Ask AI (Gemini assistant) — OPTIONAL

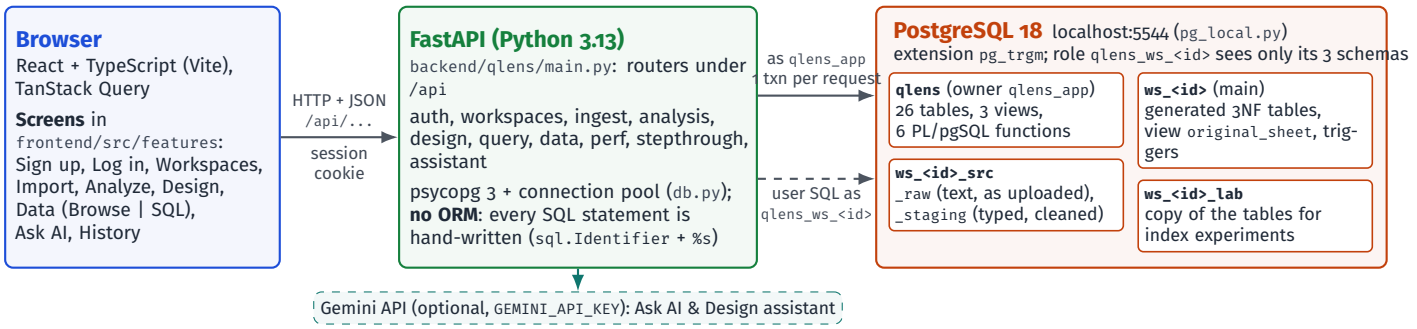
Off when `GEMINI_API_KEY` is not set; everything else works without it. Ask AI page (read-only, context `'data'`) and the Design assistant (may rename, accept/reject FDs, rebuild, undo). The model never touches the database directly: it can only call the 10 tools of `assistant/tools.py` (`describe_schema`, `check_dependency` → `check_fd`, `run_sql` → `validate_select` + sandbox, `explain_query`, `rename_table`, ...). Stored in `assistant_conversations` → `assistant_messages` → `assistant_tool_calls` (`005`); each edit is a new draft (`based_on_id`), so Undo works.

Syllabus topics QLens does not implement itself

Hashing / hash indexes, RAID, distributed and parallel databases, timestamp-ordering protocol and recovery code are PostgreSQL's job or out of scope (QLens uses B-tree and GIN indexes, row locks, and PostgreSQL's WAL). Multivalued dependencies (4NF) are not detected. Generalization appears only as type attributes. "NoSQL" side: JSONB columns (`row_changes.before/after`, `explain_snapshots.plan`).

PART B — How the code works

stack · tables · request walkthroughs · file map



Which DBMS and libraries

- **PostgreSQL 18**, run as a private local cluster (`backend/.pgdata`, port 5544) by `pg_local.py` (`initdb`, `pg_ctl`); databases `qlens` and `qlens_test` (pytest).
- **psycopg 3** driver + `psycopg_pool`; **no ORM**. **pglast** = PostgreSQL's own parser, used to check typed SQL and split it for step-through. **pg_trgm** = `similarity()`, the `%` operator and GIN trigram indexes.
- Migrations `backend/sql/migrations/000–008` run in order by `migrate.py`: 000 bootstrap · 001 tables, constraints, indexes · 002 triggers + views · 003 `try_date` · 004 `check_fd`, `is_unique` · 005 assistant tables, design versions · 006 `row_changes` + audit trigger · 007 SECURITY DEFINER, REVOKE · 008 `has_renames`.

Per-workspace schemas (the user's data)

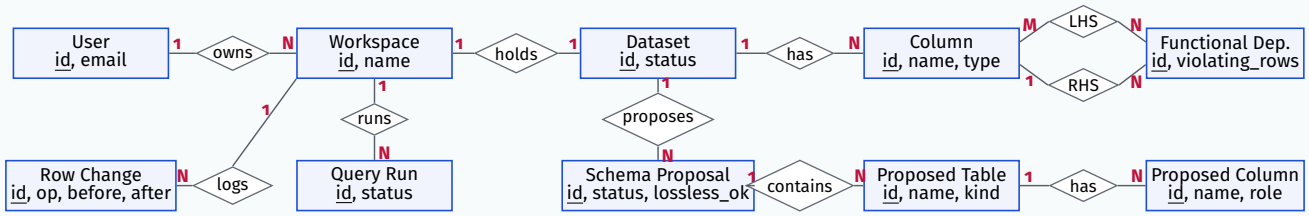
- **`ws_<id>_src`**: `_raw` = every sheet column as `text` (+ `_row_id` PK), loaded by `COPY`, never changed; `_staging` = typed copy (`CREATE TABLE AS SELECT`) that clean-up edits and analysis reads.
- **`ws_<id>`** (main): the applied tables, for the fee register:

```
teachers(id, teacher U, teacher_phone U)
batches(id, batch U, teacher_id→teachers)
students(id, student_name U, student_phone U,
         batch_id→batches)
fee_register(id, student_id→students, month, fee_paid,
             payment_mode; U(student_id, month))
```

plus view `original_sheet` and trigger `log_row_change` on each table (U = UNIQUE; FKs are `ON DELETE RESTRICT`).

- **`ws_<id>_lab`**: copy of the applied tables (`CREATE TABLE ... (LIKE ...)`) for Explain & speed up and "scale ×N"; rebuilt after each Apply (`perf/lab.py:ensure_lab L123`).

ER sketch of the `qlens` schema (textbook notation, main entities)



Rectangles = entity sets, diamonds = relationship sets, 1/N/M = cardinality. LHS is M:N because a dependency may have two columns on its left side (table `fd_lhs_columns`). Not drawn: Saved Query 1:N Query Run, Candidate Key M:N Column, Proposed Column *refers to* Proposed Table, the 3 assistant tables.

The 26 tables of schema `qlens`: keys and references

PK underlined · U = UNIQUE · C = ON DELETE CASCADE · N = SET NULL · `x_id` refers to table `xs` unless named

table	keys	references
Accounts		
`users`	id; U(email)	–
`sessions`	token_hash	user_id C
`workspaces`	id; U(owner_id, name)	owner_id→users C
Imported sheet		
`datasets`	id; U(workspace_id): 1:1	workspace_id C
`dataset_columns`	id; U(dataset_id, name), U(dataset_id, ordinal)	dataset_id C
Analysis		
`candidate_keys`	id	dataset_id C
`candidate_key_columns`	key_id, column_id: M:N	key_id C, column_id C
`functional_dependencies`	id	dataset_id C, rhs_column_id C
`fd_lhs_columns`	fd_id, column_id: M:N	fd_id C, column_id C
`duplicate_groups`	id; U(dataset_id, column_id, canonical_value)	dataset_id C, column_id C
`duplicate_values`	group_id, value: weak entity	group_id C
`dataset_fixes`	id	dataset_id C, column_id C
`row_changes`	id (written by the trigger)	workspace_id C

table	keys	references
Designs		
`schema_proposals`	id; partial U: 1 draft + 1 applied	dataset_id C, based_on_id N
`proposed_tables`	id; U(proposal_id, name)	proposal_id C
`proposed_columns`	id; U(table_id, name)	table_id C, references_table_id C, source_column_id N
`proposed_unique_keys`	id	table_id C
`proposed_unique_key_columns`	key_id, column_id: M:N	key_id C, column_id C
Queries and performance		
`saved_queries`	id; U(workspace_id, title)	workspace_id C
`query_runs`	id	workspace_id C, saved_query_id N
`nl_questions`	id; U(query_run_id)	query_run_id C (not yet written)
`explain_snapshots`	id	workspace_id C, query_run_id C
`index_experiments`	id	workspace_id C
Assistant (optional)		
`assistant_conversations`	id	workspace_id C
`assistant_messages`	id; U(conversation_id, seq)	conversation_id C
`assistant_tool_calls`	id; U(message_id, seq)	message_id C, query_run_id N

Named parents: `rhs_column_id`, `source_column_id`, `column_id` → `dataset_columns` (in `proposed_unique_key_columns`: → `proposed_columns`); `owner_id` → `users`; `based_on_id` → `schema_proposals`; `table_id`, `references_table_id` → `proposed_tables`; `key_id` → `candidate_keys` / `proposed_unique_keys`; `fd_id` → `functional_dependencies`; `group_id` → `duplicate_groups`; `message_id` → `assistant_messages`.

Walkthrough 0: create a workspace (dashboard)

1. `POST /api/workspaces` → `workspaces/service.py:create_workspace L71`.
2. `INSERT INTO qlens.workspaces ... ON CONFLICT (owner_id, name) DO NOTHING RETURNING id`.
3. `CREATE ROLE qlens_ws_<id> ...`, `ALTER ROLE ... SET ...`, `CREATE SCHEMA` + GRANTs for `ws_<id>` and `ws_<id>_src`, all in the same transaction.
4. List: `GET /api/workspaces` reads `qlens.v_workspace_summary`. Delete drops the 3 schemas, `DROP OWNED BY`, `DROP ROLE`, then the row (cascades).

Walkthrough 1: Import

1. `POST /api/workspaces/{id}/dataset` (file) or `/dataset/sample` → `ingest/routes.py:upload_dataset`, `import_sample`.
2. `ingest/parse.py:read_table`: CSV/XLSX → rows; headers become safe names (`sanitize_names`).
3. `ingest/stage.py:import_dataset L62`, one transaction: `_reset_workspace` (DELETE old dataset, cascades; DROP/CREATE schemas) → `_load_raw` (CREATE TABLE _raw, COPY) → `_infer_type` → `_build_staging` → `_record` (INSERT `datasets`, `dataset_columns` with null/distinct counts).
4. `GET .../dataset` shows the first 50 rows of `_staging`.

Walkthrough 2: Analyze (and clean-up)

1. `POST .../analyze` → `analysis/discover.py:analyze L155`, one transaction.
2. DELETE old keys/FDs; `_find_keys` (`qlens.is_unique`), `_find_fds` (`qlens.check_fd`) → INSERT `candidate_keys`, `candidate_key_columns`, `functional_dependencies`, `fd_lhs_columns`.
3. `cleanup.py:find_duplicates`: `similarity()` + GIN trigram index → `duplicate_groups`, `duplicate_values`.
4. `UPDATE qlens.datasets SET status = 'analyzed'`.
5. Fixes: `POST .../duplicates/{g}/merge`, `.../inconsistencies/{fd}/fix`, `.../cleanup/accept-all` → UPDATE `_staging` (FOR UPDATE), INSERT `dataset_fixes`, re-analyze in the same transaction. `PATCH .../fds/{id}` sets `accepted`.

Walkthrough 3: Design

1. `POST .../proposal` → `design/proposal.py:build L186`.
2. `discover.load_fds` → `to_algebra` (exact, accepted, not derived) → `fd_algebra.synthesize_3nf` → `_plan` (names, `id`, FKs, UNIQUE keys).
3. Transaction: old draft → `'superseded'`; INSERT `schema_proposals`, `proposed_tables`, `proposed_columns`, `proposed_unique_keys` (+ `_columns`).
4. Renames: `PATCH .../proposal/tables/{id}`, `.../columns/{id}` (row lock `FOR UPDATE`). SQL panel: `GET .../proposal/sql` → `ddl.render` + `ddl.as_text`.

Walkthrough 4: Apply design

1. `POST .../apply` → `design/apply.py:apply_proposal L67`.
2. One transaction: lock the draft (`FOR UPDATE`); `DROP SCHEMA ws_<id>_lab, ws_<id> CASCADE`; `CREATE SCHEMA`; run `ddl.render`: CREATE TABLE (parents first), INSERT … SELECT, CREATE TRIGGER, CREATE VIEW `original_sheet`, verify (EXCEPT ALL).
3. Rows differ → 409 `lossless_check_failed` with sample rows, ROLLBACK: old tables stay.
4. Else UPDATE `schema_proposals` (old applied → superseded, this → applied, `lossless_ok = true`), `datasets.status = 'applied'`; then `ANALYZE` each table.

Walkthrough 5: Data > Browse

1. `GET .../data/{table}?page&sort&dir&q&f` → `data/service.py:list_rows L243`: `SELECT count(*)` + `SELECT ... WHERE ... ORDER BY ... LIMIT %s OFFSET %s`.
2. `POST/PATCH/DELETE .../data/{table}[/{row_id}]` → `insert_row`, `update_row`, `delete_row` inside a savepoint (`_guarded`); a constraint error → `_explain` reads `pg_constraint` and returns a "constraint card".
3. Trigger → `qlens.row_changes`; `GET .../data-changes` = audit timeline. Also `/related` (rows pointing here), `/export` (CSV), `/options` (FK picker).

Walkthrough 6: Data > SQL

1. SELECT: `POST .../query` → `query/routes.py:run_query L162` → `validate_select` → `sandbox.run` (role, READ ONLY, cursor) → `record_run`: INSERT `qlens.query_runs` (trigger updates `saved_queries`).
2. Write (after the confirm dialog): `POST .../query/write` → `validate_write` → `sqlwrite.writable_table` → `sandbox.run_write` (READ WRITE, COMMIT); audit trigger fires.
3. `POST .../stepthrough` → `stages.run_stages`. `POST .../explain` → `EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)`, `explain_snapshots`, `advisor.suggest`; `POST .../lab/try-index` → `lab.try_index`.

Walkthrough 7: Ask AI — OPTIONAL

1. `POST .../assistant` (`context 'data'`) → `assistant/service.py:respond L134`.
2. INSERT `assistant_conversations` (new chat) and the user's `assistant_messages` row.
3. Up to 6 rounds: Gemini (`assistant/llm.py`) asks for tools → `tools.execute`, e.g. `run_sql` = `validate_select` + `sandbox.run` + a `query_runs` row (`source 'english'`).
4. INSERT the reply + `assistant_tool_calls`. A Gemini error rolls the whole turn back (429 = rate limit).

Walkthrough 8: History

1. `GET .../history` → `SELECT ... FROM qlens.query_runs WHERE workspace_id = %s ORDER BY created_at DESC, id DESC LIMIT %s` (uses `query_runs_workspace_time_idx`).
2. `GET .../slow-queries` → `v_slow_queries`; `.../saved-queries` (CRUD) → `saved_queries`; `.../saved-queries/{id}/trend` → `v_saved_query_trend`.

File map: backend/qlens/

`main.py` FastAPI app, routers under `/api`, opens/closes the pool.
`config.py` settings from `.env` (database URL, secret key, optional Gemini key).
`db.py` pool; `get_conn` = 1 transaction per request; workspace role/schema helpers, GRANTs.
`pg_local.py` private PostgreSQL 18 cluster (init/start/stop/status).
`setup_db.py` role `qlens_app`, databases, `pg_trgm`. `migrate.py` runs the migrations. `errors.py` `ApiError` → JSON error.
`auth/` sign-up, log-in, sessions; `deps.py`: `current_user`, `owned_workspace`.
`workspaces/` create (role + schemas), list (view), delete.
`ingest/` `parse.py` CSV/XLSX; `stage.py` `_raw`, `_staging`, types, profile.
`samples/generate.py` 3 sample sheets with planted mistakes.
`analysis/` `discover.py` keys + FDs; `fd_algebra.py` closure, canonical cover, 3NF; `cleanup.py` duplicates, fixes.
`design/` `proposal.py` build/store/rename/versions; `naming.py` table names; `ddl.py` the SQL script; `apply.py` run it + lossless check; `catalog.py` reads `pg_catalog`.
`query/` `validate.py` SQL checks (pglast); `sandbox.py` role connection, cursor, timeouts; `routes.py` run, write, history, saved queries; `serialize.py` values → JSON.
`data/` `service.py` Browse rows, edits, related rows, CSV, audit log; `sqlwrite.py` typed writes.
`perf/` `explain.py` plan tree; `advisor.py` index suggestions; `lab.py` lab copy, scale, try index.
`stepthrough/stages.py` SELECT → algebra stages. `assistant/` `llm.py` Gemini client, `tools.py` the 10 tools, `service.py` chat loop + undo.
`backend/tests/` 26 pytest files against a real PostgreSQL test database (e.g. `test_fd_algebra.py`, `test_apply.py`, `test_sandbox.py`, `test_validate.py`).

Frontend `frontend/src/features/`: `auth`, `workspaces` (dashboard, step rail), `import`, `analyze`, `design` (ER canvas, SQL panel, Apply), `data` (Browse, SQL tab, constraint cards), `query` (editor, results, step-through), `perf` (plan, before/after), `assistant` (Ask AI, design assistant), `history`; `lib/api.ts` = fetch wrapper.

Viva quick answers

Why surrogate keys (`id`)? A short number that never changes when a name is corrected, so foreign keys never need updating and joins use one column. The natural key is still enforced by UNIQUE.

How do you prove the decomposition is lossless? Theory: each join is on a key of one side and one table holds a candidate key. Practice: Apply rebuilds the sheet with `original_sheet` and runs EXCEPT ALL both ways; both counts must be 0, else ROLLBACK.

How do you stop SQL injection? Values are bound parameters (`%s`); names go through `sql.Identifier` / `format('%I')`; typed SQL must parse as one SELECT and runs as a role that can read only its own workspace.

Why a trigger for the audit log? The database records every change from every path (forms, typed SQL) in the same transaction; code cannot forget it, and a rolled-back change leaves no log row.

What is the cursor for? The sandbox DECLAREs a cursor for the user's SELECT and FETCHes at most 1,001 rows, so a huge result never fills memory; the 1,001st row only signals "truncated".

Why compute a canonical cover first? It removes redundant FDs and extra left-side columns, so the 3NF algorithm makes no needless tables (no table for student_name → teacher).

Why 3NF and not BCNF? 3NF synthesis is always lossless *and* dependency preserving; BCNF can lose dependency preservation. The fee register tables are in BCNF anyway.

Where are FDs found: Python or SQL? In SQL: `qlens.check_fd` groups the rows inside PostgreSQL. Python only chooses which columns to test and runs closure, cover and synthesis on the small list of FDs.

Two people click Apply at once? The draft row is locked with `SELECT ... FOR UPDATE`; the second waits, sees `'applied'` and gets 409. Partial unique indexes allow one draft and one applied design per dataset.

Why `lossless_ok IS TRUE` in the CHECK? A CHECK passes on NULL; `IS TRUE` makes "not checked yet" (NULL) fail.

Why one schema and role per workspace? Isolation and least privilege: a user's SQL can see only their own tables, and `DROP SCHEMA ... CASCADE` removes a workspace in one statement.

What happens on a constraint error in Browse? Only the savepoint is rolled back; QLens reads `pg_constraint` and shows which rule (UNIQUE, FOREIGN KEY, NOT NULL, CHECK) refused the change.

Demo order for the viva (from the README)

1. Sign up → dashboard → **Try a sample** → fee register (3,000 rows).
2. **Analyze**: exact FDs (student_name → batch, batch → teacher), the candidate key, inconsistencies (student_phone → student_name broken by 8 rows) → fix → **Accept all**.
3. **Design**: the 3NF tables on the ER canvas; rename a table; **Apply design** → lossless check (EXCEPT ALL both ways = 0 rows).
4. **Data** > **Browse**: add a student with an existing name (UNIQUE card); delete a student who has fee rows (FOREIGN KEY card, `ON DELETE RESTRICT`). In **Data** > **SQL**, confirm:

```
INSERT INTO fee_register (student_id, month, fee_paid, payment_mode)
VALUES (999999, 'Apr', 4000, 'UPI');      -- no student 999999: FK card
```

5. **Data** > **SQL**: a join; `SELECT * FROM fee_register WHERE fee_paid = 2100` → **Explain & speed up** (Seq Scan, try the index); a GROUP BY query → **Step through**; an UPDATE with the confirm dialog.
6. **Ask AI** (needs the Gemini key, else it says why it is off), then **History**.